***Володимирський педагогічний фаховий коледж імені Агатангела Кримського***
циклова комісія викладачів професійної цифрової та математичної освіти

# Інструкція до практичної роботи №58

# ТЕМА: Створення векторних графічних елементів та кастомних схем у середовищі CustomTkinter.

*Освітньо-професійна програма: Професійна освіта (Цифрові технології)*
*Тривалість заняття: 80 хв*

**Мета заняття:**

- закріпити навички роботи з графічними примітивами CTkCanvas;
- навчитися компонувати кілька примітивів у складніші, змістовні графічні елементи;
- ознайомитися з побудовою власних векторних схем (блок-схем, діаграм, значків);
- розвинути навички планування малюнка «на папері» перед його реалізацією в коді.

## Теоретичні відомості

### *1. Від примітивів до змістовних зображень*

Жоден зі складних малюнків, які ми звикли бачити в реальних застосунках (іконки, схеми, діаграми), не є єдиним «спеціальним» об'єктом — усі вони, як і в практичній роботі №57, будуються комбінацією простих примітивів (прямокутників, овалів, ліній), розташованих у потрібних місцях і з потрібними кольорами. Уміння розкладати складне зображення на прості складові частини — головна навичка векторної графіки, яка знадобиться і в цій, і в подальших практичних роботах.

### *2. Планування малюнка перед написанням коду*

Перш ніж писати команди create_..., корисно спочатку намалювати ескіз майбутньої схеми на папері (або хоча б подумки), визначивши: скільки й яких фігур знадобиться; приблизні координати кожної фігури; кольори кожного елемента. Такий попередній етап значно скорочує кількість спроб «підбору» координат безпосередньо в коді методом проб і помилок.

### *3. Приклад: побудова простої блок-схеми*

Розглянемо побудову класичної блок-схеми алгоритму (овал-початок, прямокутник-дія, овал-кінець), з'єднаних стрілками — подібну схему ми неодноразово бачили в темі 2, але тепер побудуємо її не готовим зображенням, а власноруч засобами CTkCanvas:

```
canvas = ctk.CTkCanvas(app, width=380, height=300, bg='#242424',
highlightthickness=0)
```

```
canvas.pack(padx=10, pady=10)

canvas.create_oval(140, 15, 240, 55, fill='#3a7ebf',
outline='white', width=2)
canvas.create_text(190, 35, text='Початок', fill='white',
font=('Arial', 11))

canvas.create_line(190, 55, 190, 90, fill='white', width=2,
arrow='last')

canvas.create_rectangle(120, 90, 260, 130, fill='#2a9d8f',
outline='white', width=2)
canvas.create_text(190, 110, text='Обробка даних', fill='white',
font=('Arial', 11))

canvas.create_line(190, 130, 190, 165, fill='white', width=2,
arrow='last')

canvas.create_oval(140, 165, 240, 205, fill='#3a7ebf',
outline='white', width=2)
canvas.create_text(190, 185, text='Кінець', fill='white',
font=('Arial', 11))
```

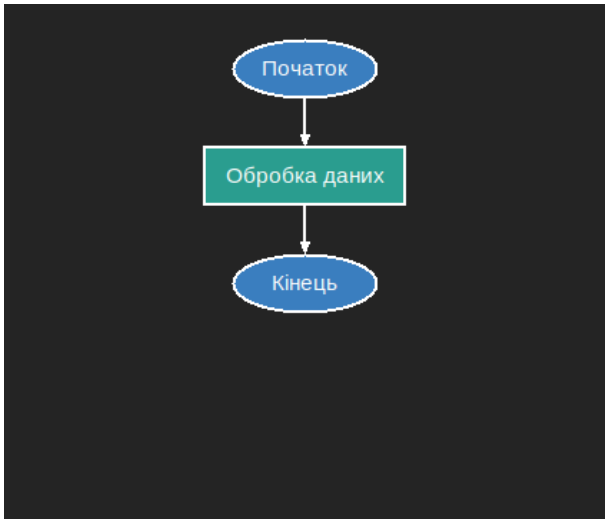



*Рис. 1. Проста блок-схема, побудована власноруч комбінацією овалів, прямокутника, ліній та тексту*

Зверніть увагу на параметр arrow='last' у методі create_line() — він додає до кінця лінії наконечник стрілки, що є стандартним і дуже зручним способом показати напрямок з'єднання елементів схеми, не малюючи трикутник наконечника вручну окремою фігурою.

### *4. Прийом вирівнювання елементів за спільною віссю*

У прикладі 3 усі елементи (овали, прямокутник) навмисно мають однакову координату центру по осі x (190) — так забезпечується їх акуратне вертикальне вирівнювання одне під одним. Такий прийом — заздалегідь визначити спільну

«вісь» (значення x або y), відносно якої вирівнюються всі елементи схеми, — істотно полегшує побудову охайних, симетричних композицій.

### *5. Побудова власного «значка» (іконки) з примітивів*

Іншим поширеним застосуванням векторної графіки є побудова простих піктограм — наприклад, значка «додому», «пошук» чи «кошик» — з елементарних фігур. Розглянемо спрощений значок будиночка:

```
def draw_home_icon(canvas, x, y, size, color):
    canvas.create_polygon(
        x, y - size, x - size, y, x + size, y,
        fill=color, outline=''
    )
    canvas.create_rectangle(
        x - size * 0.6, y, x + size * 0.6, y + size,
        fill=color, outline=''
    )

draw_home_icon(canvas, 100, 150, 40, '#f2cc8f')
```

Оформлення побудови значка у вигляді функції з параметрами (координати центру, розмір, колір) — важливий, зручний прийом: він дозволяє намалювати такий значок будь-де на полотні, будь-якого розміру й кольору, викликаючи функцію з різними аргументами, замість дублювання координат вручну для кожного нового значка.

### *6. Композиція кількох незалежних елементів схеми*

Складніші схеми зазвичай складаються з кількох незалежних груп фігур (наприклад, кількох значків поруч, або кількох блоків схеми, не пов'язаних стрілками). У такому разі кожну групу зручно будувати власною функцією, а в основній частині програми лише викликати ці функції з потрібними координатами зсуву — це та сама ідея декомпозиції задачі на функції, яку ми детально розглядали в темі 4, застосована тепер до графіки.

## Хід роботи

### *Варіант і вимоги до здачі*

а) N — ваш порядковий номер у списку групи (журнал). Дані залежать від N (mod — остача від ділення):

| Завдання | Дані варіанта | Приклад N = 7 |
|---|---|---|
| 5 | кількість підлеглих 3 + N mod 2 | 4 |
| 6* | умова «x > N?» | x > 7? |

б) Кожне завдання — окремий файл Прізвище_ПР58_зN.py (наприклад, Ivanenko_PR58_z3.py), код текстом, а не фото чи скріншотом. Файли з

прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного завдання додайте скріншот вікна із видимим заголовком. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів і повідомлень — точно як в умові. Фігури малюйте лише примітивами CTkCanvas; координати зв'язаних блоків обчислюйте відносно один одного, а не підбирайте «на око».

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР58_з1.py. Накресліть на папері ескіз схеми з 3–4 блоків («Вхід» → «Обробка» → «Вихід») і реалізуйте її на CTkCanvas: прямокутники з підписами create_text(). Додайте фото ескізу до звіту.

   **Приклад:** 3 прямокутники в ряд, підписи по центру блоків.

2. З'єднайте блоки лініями create_line(..., arrow='last') так, щоб стрілки показували напрямок зв'язків.

   **Приклад:** стрілка від кожного блока до наступного.

3. Напишіть функцію draw_home_icon(canvas, x, y, size, color) (як у прикладі 5) і намалюйте нею три будиночки розмірів 40, 60, 80 різних кольорів у різних місцях полотна.

   **Приклад:** розміри та кольори різні, будиночки не накладаються.

4. Напишіть власну функцію значка (зірка, конверт чи стрілка) з параметрами canvas, x, y, size, color і викличте її двічі.

   **Приклад:** значок масштабується разом із size.

5. Побудуйте організаційну схему: один «керівник» зверху й 4 «підлеглих» знизу, з'єднаних лініями; блоки в нижньому ряду розташуйте рівномірно.

   **Приклад (N = 7):** 4 підлеглих; кількість підлеглих = 3 + N mod 2.

6. Завдання підвищеної складності*. Блок-схема з умовою «x > 7?»: овал «Початок», прямокутник «Введіть x», ромб (create_polygon із 4

вершинами) з виходами «Так» → «Більше 7» і «Ні» → «Не більше 7», спільний овал «Кінець»; усе — лише примітивами CTkCanvas.

**Приклад (N = 7):** ромб «x > 7?»; гілки підписані «Так» і «Ні».

## Критерії оцінювання

| Складова | Оцінювання | Макс. балів |
|---|---|---|
| Завдання 1–5 | кожне по 1,8 бала | 9 |
| Завдання 6* | 2 бали | 2 |
| Контрольні питання | відповіді у звіті | 1 |
| **Разом** | | **12** |

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, стрілки не відповідають напрямку, функція значка не приймає параметри, використано готове зображення замість примітивів, не підписано гілки ромба) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

### *Захист роботи*

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити розмір чи кількість блоків схеми). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

## Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чому перед написанням коду складної схеми корисно спочатку намалювати ескіз на папері?
2. Який параметр методу create_line() додає наконечник стрілки в кінці лінії?
3. У чому перевага оформлення побудови значка у вигляді функції з параметрами координат, розміру й кольору?
4. Що означає «вирівнювання елементів за спільною віссю» і чим воно корисне при побудові схем?

5. Наведіть приклад складнішої схеми, яку зручно розбити на кілька незалежних функцій-«блоків».